AI-Powered IMU Gesture Controller for Hands-Free Desktop and Presentation Control

Jordan Wray

Jwray2021@fau.edu

This Research Report is being submitted as a response to the Final Project on Generative Intelligence Explorations, part of COT6930 Generative Intelligence and Software Development Lifecycles, instructor Dr. Fernando Koch.

- ☐ **Track 1 - Research**
- ☑ ~~**Track 2 - Application**~~
- ☐ **Track 3- Blue Sky Idea**

Link to Implementation: https://github.com/Jordan122401/imu-gesture-smart-remote (Demo and all code used is in this github repo)

AI Assistance Note: The project idea, dataset collection, implementation, testing, and rough draft were completed by the student. Generative AI was used only to refine wording, organize code, and improve clarity in the final draft.

Abstract

This project looks at the problem of building a hands-free desktop and presentation controller using AI. Normally, controlling a desktop or a slideshow relies on a keyboard, touchpad, or handheld clicker, all of which still need direct physical contact. What I'm exploring here is IMU-based gesture recognition with deep learning as a practical alternative. The system uses an ESP32 paired with an MPU6050 to capture multivariate time-series motion data, and a trained neural network then classifies each gesture and maps it to a desktop action.

The current prototype builds directly on the pipeline I put together for Assignment 2, which already covers data collection, preprocessing, model training, and live inference. For the final project direction, I'm extending that work into an actual gesture controller application. The main hardware change at this checkpoint is a physical button added to the ESP32 for controlled gesture capture. Basically, press-to-record instead of relying on motion thresholds alone. The prototype right now is focused on gesture-based desktop and slide control, with extra actions like volume up/down planned for the final version. Early results suggest the project is technically feasible and that the existing AI pipeline can be extended into a real-time application without too much trouble.

Introduction

The key question guiding this project is whether an IMU-based deep learning system can be turned into a practical gesture controller for desktop and presentation tasks.

While exploring the problem of hands-free desktop and slideshow control, I observed that existing methods may struggle with staying out of the user's way. Keyboards, touchpads, and handheld clickers all still require direct physical contact, which breaks the natural flow of a presentation or demo.

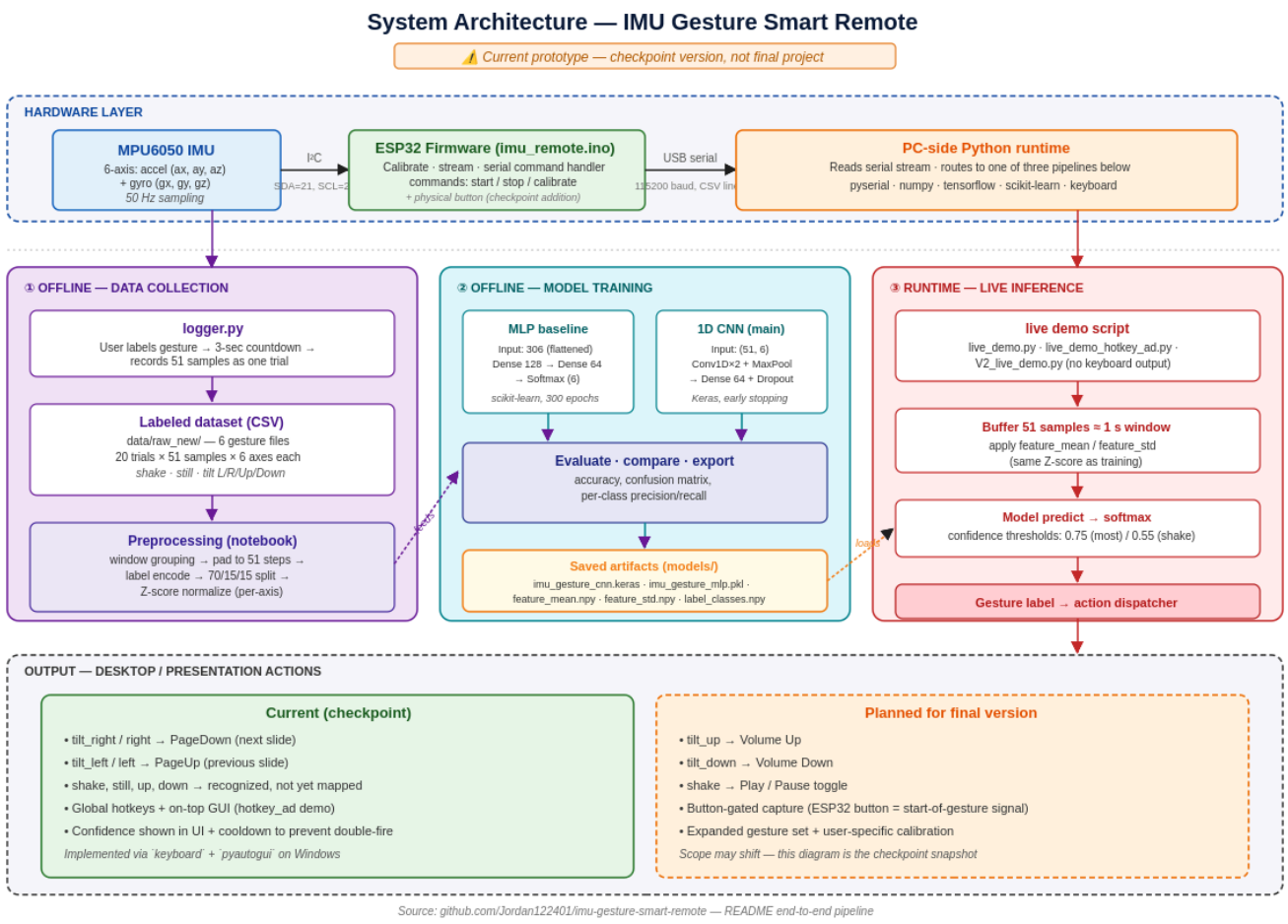
This limitation makes it difficult to effectively address situations where the user wants to trigger commands without reaching for a device, such as giving a talk, moving between slides, or adjusting volume while their hands are occupied.

To explore this idea, I designed a small experimental study using an ESP32 and MPU6050 IMU to capture 6-axis motion data, a 1D CNN (with an MLP as a baseline) trained on fixed-length windows of that data, and a live Python inference loop that maps predicted gestures to desktop actions. A physical button was added to the ESP32 to support controlled gesture capture during recording and testing.

The goal of this project is to better understand how this AI technique behaves and how it can be applied to the selected problem.

Our preliminary observations suggest that the approach is technically doable. The existing pipeline from Assignment 2 extends cleanly into a real-time controller, the classifier runs fast enough for live use, and the button-based capture makes the gesture boundaries much cleaner than relying on motion thresholds alone. Expanding the gesture-to-command mapping (volume control and a few extra actions) looks straightforward from here.

Proposal



Source: github.com/Jordan122401/imu-gesture-smart-remote — README end-to-end pipeline

Illustrate the overall workflow of the system or AI model.

In this project I proposed a system that uses an ESP32 with an MPU6050 6-axis IMU on the sensor side and three small neural networks on the PC side, a simple MLP, and a 1D CNN to handle the classification.

The main idea behind this approach is to treat each hand gesture as a short time series of about a second of accelerometer and gyroscope data and then see how different types of networks deal with it as we change the amount of training data. I'm not really chasing peak accuracy here, since earlier work already shows you can get near-perfect results on this kind of data once you have enough samples. What I actually want to know is which architecture holds up the best when the user can only give a few examples, because that's what an end user is realistically going to do in practice.

The proposed solution can be described as a workflow consisting of the following steps: first, the MPU6050 streams 6-axis motion data to the ESP32 at 50 Hz over I²C. The ESP32 firmware calibrates the readings and sends them to my laptop over USB serial at 115200 baud. On the PC, a logger script records labeled trials, each gesture becomes a 51-sample window, which is roughly one second of motion. A Jupyter notebook then handles the preprocessing (windowing, padding, label encoding, a 70/15/15 split, per-axis Z-score normalization) and trains all three models under the same conditions. To actually answer the research question, each model is retrained on different fractions of the training data (10%, 25%, 50%, 75%, 100%), with several random seeds per setting, so I end up with a proper data-efficiency curve instead of just a single accuracy number. Whichever model comes out on top gets used in the live demo, which reads the serial stream, buffers each new window, applies the same normalization, and fires off a keyboard shortcut when the model is confident enough.

The main components of the system include the MPU6050 sensor, the ESP32 running my custom firmware (including a physical button for controlled gesture capture), the USB-serial link, the data collection pipeline, the three trained models with their saved weights and normalization stats, the evaluation code that runs the data-efficiency sweep, and the live inference scripts that turn predictions into actual desktop actions.

This design allows the system to process the input data and produce predictions or classifications by using the same normalization at inference time that I computed on the training set, then pushing each 1-second window through the trained network. The system only acts on the prediction if the softmax confidence is above a threshold (0.75 for most gestures, lower for "shake" since it's noisier), and a short cooldown keeps the same gesture from firing twice in a row. On the research side, the training code is wrapped in a single function that takes the model type, data fraction, and random seed as inputs, which makes the whole data-efficiency experiment reproducible and easy to extend if I add more architectures later.

Experiments

Experiment 1 – MLP Baseline Compared to 1D CNN

This experiment assesses if a 1D CNN enhances IMU gesture classification relative to a simple MLP baseline. I carried out the experiment by training both models on the same gesture dataset that I collected with the ESP32 and MPU6050 sensor. Each gesture trial was saved as a fixed window of 51 samples. For each sample, there were six IMU features, namely ax_g, ay_g, az_g, gx_dps, gy_dps and gz_dps. The gesture labels included still, tilt left, tilt right, tilt up, tilt down and shake.

For the MLP model, each 51 x 6 gesture window was converted to a long input vector. This provided a simple baseline model for comparison. The 1D CNN model took the input as a time series so that the model could learn patterns over the motion window and not simply treat the data as a long list of numbers.

The models, tools, datasets and frameworks used are Python, NumPy, Pandas, Scikit-learn, TensorFlow/Keras and my self collected ESP32 and MPU6050 gesture dataset.

The experiment produced good results for both models. The MLP could classify the gestures well, indicating that the IMU features contained useful information for recognizing motion patterns. The 1D CNN also performed very well, and was selected as the main model for the final live demo because it better fits with the data structure. The CNN is more suitable than just using a flattened input as the gesture data is based on motion over time.

The results show that IMU data can be used to learn gesture classes. They also indicate that the 1D CNN is the better model for the final system as it can learn short time based patterns from the accelerometer and gyroscope signals.

Experiment 2 – Fixed Window Gesture Classification

This experiment tests if a fixed length gesture window improves the consistency of live AI prediction. The experiment was done by collecting each gesture as a fixed window of 51 sensor samples. This was important because the AI model requires the same input shape every time it makes a prediction. The system does not record gestures of arbitrary length. Instead, it records one controlled window after the user presses the physical button. The same 51 sample format is used during training and during the live desktop demo.

The models, tools, datasets and frameworks used are ESP32, MPU6050, Python serial communication, NumPy preprocessing, TensorFlow/Keras and the trained 1D CNN model.

This experiment resulted in a more robust input pipeline. It was easier to control both the training and the live inference, as the model could be fed the same input shape every time. This also made the desktop app more predictable since every button press would result in one gesture sample, one prediction and one command.

The results of this study suggest that fixed window collection is a good design choice for this project. Keeps the AI input consistent and avoids confusion during the live demo. This also makes the system easier to explain, as one button press equals one recording window for a gesture.

Experiment 3 – Live AI Prediction With Gesture to Command Mapping

This experiment tests whether the trained AI model can be used in a real desktop control application. The experiment was carried out by exporting the trained 1D CNN model and loading it into a Python-based desktop application. The ESP32 and MPU6050 collected one fixed 51 sample gesture window after physical button was pressed. Next, the python app did the same preprocessing as in training and provided the data to the trained model, checked the confidence of the prediction, and mapped the predicted gesture to a desktop command.

The models, tools, datasets and frameworks that are being used are 1D CNN model trained, label classes saved, normalization files saved, Python, Tensorflow/keras, NumPy, serial communication, Tkinter for desktop app interface and keyboard or volume control libraries for sending commands to the computer.

The experiment resulted in a working live prototype. The system could recognize gestures and use them to control the computer. The final mapping of commands was:

Gesture	Desktop Action
tilt left	previous slide
tilt right	next slide
tilt up	volume up
tilt down	volume down

shake	escape or cancel
still	no action

Results and Discussion

Include points like:

- The CNN was selected because it preserves the time-series structure of the IMU data.
- The MLP still worked well, showing the sensor features are strong.
- The fixed 51-sample window made live inference predictable.
- Button-triggered capture reduced false triggering compared to always-listening motion detection.
- The small dataset worked because the gestures were physically distinct.
- The main weakness is that the dataset was collected by one person under controlled conditions.
- Future work should test more users, more sessions, Bluetooth/BLE, and possibly on-device inference.

Experiment	Model / Setup	Result
1	MLP full dataset	100% test accuracy
1	CNN full dataset	100% test accuracy
1	MLP with 50% training data	91.67% accuracy
1	CNN with 50% training data	100% accuracy
2	Fixed 51-sample windows	Consistent input shape for training/live demo
3	Live CNN controller	Gesture prediction mapped to desktop commands